

rapport

projet dev web

SMART HOME: maison intelligente:

conception d'un site web qui gère la téléassistance des différents objets dans une maison (lumière/télévision/...)

projet réalisé par:

Ayman HAFID
Youssef MISSAOUI
Lounes-Ayoub OUBNI
Rihem SALHI

GI-Gr5



RÉSUMÉ

Dans le cadre du module de développement web, nous avons conçu une application de gestion de maison intelligente qu'on a choisi de la nommer Smart Home.

L'objectif était de créer une plateforme centralisée permettant de contrôler divers objets connectés (lampes, thermostats, caméras, ...) via une interface web sécurisée. L'application distingue deux types d'utilisateurs :

- ❖ l'administrateur, qui valide les comptes et supervise le parc d'objets,
- ❖ et l'utilisateur standard, qui interagit avec ses équipements.

Ce projet repose sur une architecture utilisant en frontend (HTML5,CSS3,...) et en backend (Python avec le framework Flask , SQLite,...)

sommaire

1. INTRODUCTION ET CONTEXTE	4
2. ANALYSE DES BESOINS	5
2.1. Besoins fonctionnels	5
2.2. Besoins non fonctionnels	6
3. TECHNOLOGIES UTILISÉES	7
4. CONCEPTION DE LA BASE DE DONNÉES	8
5. AUTHENTIFICATION ET SÉCURITÉ AVANCÉE	9
5.1. Hachage des mots de passe	9
5.2. Gestion des sessions et accès	10
6. FONCTIONNALITÉS ET CONTRÔLE	11
6.1. Interface Utilisateur	11
6.2. Module de Recherche	12
7. DÉROULEMENT DU TRAVAIL	13
7.1. méthodologie et étapes réalisées	14
7.2. répartition des tâches	15
8. CONCLUSION ET PERSPECTIVES	16

introduction et contexte

A l'ère de la transformation numérique, les objets connectés sont devenus omniprésents dans notre quotidien. On les retrouve dans les maisons, les entreprises, les moyens de transport et même dans les services de santé. Cette évolution technologique a profondément changé la manière dont les utilisateurs interagissent avec leur environnement. Aujourd'hui, il est possible de contrôler plusieurs équipements à distance, d'automatiser certaines tâches et d'améliorer le confort de vie grâce à des solutions intelligentes. Dans ce contexte, nous avons choisi de développer une application web dédiée à la maison intelligente. Ce thème nous a semblé particulièrement pertinent, car l'habitat connecté représente un domaine en pleine expansion et concerne une large catégorie d'utilisateurs. En effet, presque chaque foyer possède aujourd'hui des équipements pouvant être automatisés : éclairage, télévision, chauffage, sécurité, électroménager, etc.

Notre objectif est de proposer une plateforme simple, interactive et dynamique, capable de répondre aux besoins variés des utilisateurs modernes. L'application permet de centraliser la gestion des objets connectés à travers une interface intuitive, accessible et agréable à utiliser.

À travers ce projet, nous souhaitons répondre aux besoins de la maison de demain en rendant intelligents les objets du quotidien. Nous voulons également découvrir et appréhender la culture de la téléassistance et de l'automatisation domestique, devenue essentielle dans les sociétés modernes.

Les enjeux de ce type de solution sont nombreux. Une maison intelligente peut faciliter la vie des personnes à mobilité réduite, des personnes âgées ou dépendantes, en leur offrant davantage d'autonomie. Elle peut également aider les utilisateurs ayant un rythme de vie chargé, en automatisant certaines tâches et en leur permettant de gagner du temps.

Pour la réalisation de ce projet, nous avons utilisé plusieurs outils et technologies modernes. GitHub a été utilisé comme plateforme de collaboration et de gestion de versions. Visual Studio Code a servi d'éditeur de code principal. Le framework Flask a été choisi pour le développement backend grâce à sa simplicité et sa flexibilité. Enfin, les langages HTML, CSS et Python ont été utilisés respectivement pour la structure des pages web, le design de l'interface et la logique applicative.

analyse des besoins

2.1 Besoins fonctionnels

Pour l'utilisateur :

- ☐ Créer un compte via un formulaire d'inscription.
- ☐ Se connecter de manière sécurisée.
- ☐ Accéder à son espace personnel après validation du compte.
- ☐ Consulter la liste des objets connectés disponibles.
- ☐ Contrôler les appareils autorisés (allumer, éteindre, régler, ouvrir, fermer, etc.).
- ☐ Consulter l'état actuel des objets.
- ☐ Rechercher un objet par nom ou emplacement.
- ☐ Filtrer les objets selon leur type ou leur état.
- ☐ Se déconnecter de manière sécurisée.

Pour l'administrateur :

- ☐ Se connecter avec un compte administrateur.
- ☐ Consulter les demandes d'inscription en attente.
- ☐ Valider ou refuser un nouveau compte.
- ☐ Bloquer ou débloquer un utilisateur.
- ☐ Ajouter un nouvel objet connecté.
- ☐ Modifier les informations d'un objet.
- ☐ Supprimer un objet existant.
- ☐ Superviser l'ensemble des équipements de la plateforme.

Pour les objets connectés :

- ☐ Gérer différents types d'objets : lumière, télévision, thermostat, porte intelligente, caméra, alarme, machine à café, etc.
- ☐ Adapter les actions disponibles selon le type d'objet.
- ☐ Empêcher l'utilisation d'un objet en panne.
- ☐ Réserver certains objets sensibles ou fragiles à l'administrateur.

2.2 Besoins non fonctionnels

Ergonomie et expérience utilisateur :

- Proposer une interface claire, moderne et intuitive.
- Assurer une navigation simple entre les différentes pages.
- Offrir une interface dynamique et interactive.
- Rendre l'application agréable visuellement.

Performance :

- Charger rapidement les pages.
- Répondre efficacement aux actions utilisateur.
- Assurer un fonctionnement fluide.

Sécurité :

- Authentification sécurisée des utilisateurs.
- Protection des mots de passe par chiffrement (hashage).
- Gestion des rôles (admin / utilisateur).
- Restriction d'accès aux fonctionnalités sensibles.
- Vérification anti-robot via captcha.

Fiabilité :

- Sauvegarder correctement les données dans la base.
- Garantir la cohérence des informations.
- Éviter les erreurs critiques.

Maintenabilité et évolutivité :

- Code structuré et réutilisable.
- Possibilité d'ajouter de nouveaux objets connectés.
- Facilité de mise à jour et d'amélioration future.

technologies utilisées

➤ frontend

html5

css3

➤ backend

sql

python (en utilisant la framework
Flask)

➤ outil de collaboration

GitHUB

➤ environnement de développement

VScode

CONCEPTION DE LA BASE DE DONNÉES

Données relatives à l'application

La base de données contient principalement deux grandes catégories de données : les utilisateurs et les objets connectés.

Table **users**

Cette table permet de stocker les informations des utilisateurs inscrits sur la plateforme.

Table **devices**

Cette table permet de stocker les objets connectés de la maison intelligente.

Données de synchronisation

pour assurer la cohérence entre l'état affiché dans l'interface web et l'état enregistré dans la base de données.

À chaque action réalisée par l'utilisateur, par exemple allumer une lumière ou changer la chaîne de la télévision, l'application effectue les étapes suivantes :

- L'utilisateur choisit une action depuis l'interface.
- Flask reçoit la requête.
- L'application vérifie les droits d'accès.
- L'état de l'objet est modifié dans la base de données.
- La page est rechargée avec le nouvel état.
- L'utilisateur voit l'état mis à jour.

Cette synchronisation garantit que les informations affichées correspondent toujours aux données réellement enregistrées.

AUTHENTIFICATION ET SÉCURITÉ

5.1 Hachage des mots de passe

Lors de l'inscription, le mot de passe saisi par l'utilisateur n'est pas stocké directement dans la base de données. Pour améliorer la sécurité, nous avons utilisé un mécanisme de hachage.

Dans notre projet, le hachage est réalisé avec les fonctions proposées par Werkzeug :

- `generate_password_hash()` : utilisée lors de l'inscription pour transformer le mot de passe.
- `check_password_hash()` : utilisée lors de la connexion pour vérifier si le mot de passe saisi correspond au mot de passe haché enregistré.

5.2 Gestion des sessions et accès

Après une connexion réussie, l'application crée une **session** contenant les informations nécessaires de l'utilisateur connecté, comme son identifiant, son rôle et son nom complet.

La session permet de garder l'utilisateur connecté pendant sa navigation sur le site, sans lui demander de se reconnecter à chaque page.

Pour protéger certaines routes, nous avons utilisé des fonctions de vérification :

- `login_required` : empêche l'accès aux pages réservées aux utilisateurs non connectés.
- `admin_required` : limite certaines pages uniquement à l'administrateur.

Ainsi :

- un utilisateur non connecté ne peut pas accéder au tableau de bord ;
- un utilisateur simple ne peut pas accéder aux pages d'administration ;
- un utilisateur en attente de validation ne peut pas entrer dans le système ;
- Un utilisateur bloqué ne peut plus accéder à son compte.

Ce contrôle des accès garantit que chaque utilisateur ne peut effectuer que les actions correspondant à son rôle.

FONCTIONNALITÉS ET CONTRÔLE

6.1. Interface Utilisateur

L'interface utilisateur a été conçue pour être moderne, intuitive et facile à utiliser. Elle permet à chaque utilisateur d'accéder rapidement aux fonctionnalités essentielles sans difficulté technique.

Principales pages de l'application :

- ★ **Page d'accueil** : présente le projet et donne accès à la connexion ou à l'inscription.
- ★ **Page de connexion** : permet d'accéder à un compte existant.
- ★ **Page d'inscription** : permet de créer un nouveau compte avec validation des informations.
- ★ **Tableau de bord administrateur** : réservé à l'admin pour gérer les utilisateurs et les objets.
- ★ **Tableau de bord utilisateur** : affiche les objets disponibles avec leurs états.
- ★ **Page de contrôle d'objet** : permet d'exécuter des actions spécifiques selon le type d'objet.

Caractéristiques de l'interface :

- ★ Navigation claire entre les pages.
- ★ Boutons visibles et colorés.
- ★ Affichage dynamique de l'état des objets.
- ★ Messages de confirmation ou d'erreur.
- ★ Mise en page responsive adaptée à différents écrans.
- ★ Design moderne avec couleurs et effets visuels.

6.2. Module de Recherche

Afin de faciliter la gestion des objets connectés, un module de recherche a été intégré dans l'application.

Ce module permet de retrouver rapidement un objet spécifique parmi plusieurs équipements enregistrés.

Fonctions disponibles :

- Recherche par **nom de l'objet**
- Recherche par **emplacement** (salon, cuisine, chambre, etc.)
- Filtrage par **type d'objet**
- Filtrage par **état** ou condition

Exemples :

- Rechercher : *Lampe Salon*
- Filtrer : *Télévision*
- Filtrer : *En panne*
- Rechercher : *Cuisine*

DÉROULEMENT DU TRAVAIL

7.1. méthodologie et étapes réalisées

Pour mener à bien ce projet, nous avons adopté une méthodologie de travail progressive inspirée des approches Agile. Cette organisation nous a permis de répartir les tâches entre les membres du groupe, de suivre l'avancement du projet et d'améliorer progressivement l'application jusqu'à la version finale.

Méthode de collaboration

Pour assurer un travail efficace en groupe, nous avons utilisé :

- GitHub pour partager le code source,
- Visual Studio Code pour le développement,
- réunions et échanges réguliers pour coordonner les tâches,
- répartition des responsabilités selon les compétences de chacun.

Le développement a été réalisé en plusieurs étapes successives.

ÉTAPE 1

ANALYSE DU BESOIN

- à définir le thème du projet et identifier les besoins des futurs utilisateurs.

fonctionnalités principales :

- inscription et connexion,
- gestion des utilisateurs,
- gestion des objets connectés,
- contrôle des appareils,
- interface simple et moderne.

ÉTAPE 2

CONCEPTION DU SYSTÈME

Après l'analyse, nous avons conçu l'architecture générale de l'application

- structure des pages web,
- rôles des utilisateurs,
- organisation du code,
- conception de la base de données,
- définition des objets connectés à intégrer.

ÉTAPE 3

DÉVELOPPEMENT BACKEND

programmation de la logique principale de l'application avec Flask et Python :

- création des routes,
- gestion des formulaires,
- authentification,
- sessions,
- interactions avec la base SQLite,
- contrôle des objets connectés.

ÉTAPE 4

DÉVELOPPEMENT DE L'INTERFACE

- Nous avons ensuite travaillé sur l'interface utilisateur avec HTML et CSS
- Les différentes pages ont été reliées au backend pour assurer un fonctionnement complet.

ÉTAPE 5

TESTS ET CORRECTIONS

Une phase importante a été consacrée aux tests :

- vérification des inscriptions,
- tests de connexion,
- validation des rôles,
- ajout/modification/suppression d'objets,
- contrôle des équipements,
- correction des bugs détectés.

Cette étape a permis d'améliorer la stabilité du projet.

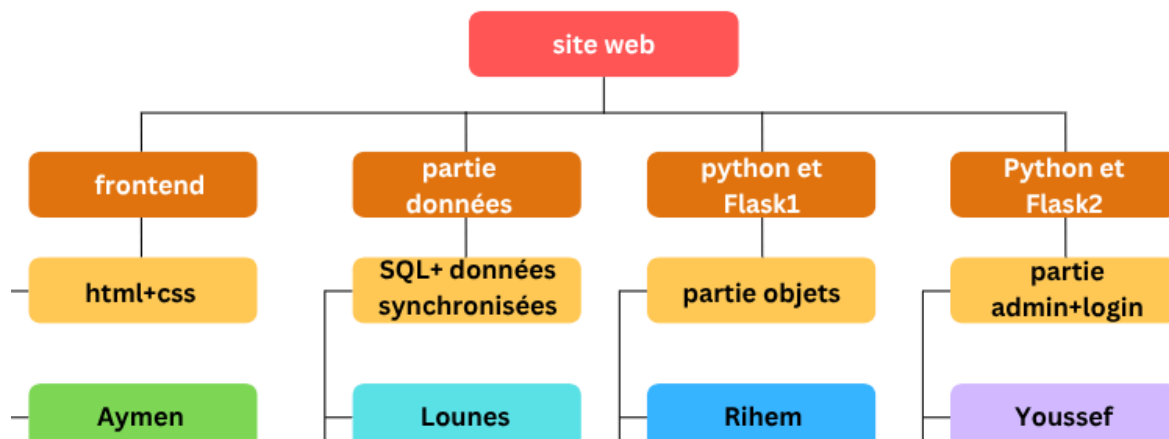
ÉTAPE 6

FINALISATION ET DOCUMENTATION

Enfin, nous avons préparé la version finale du projet :

- nettoyage du code,
- organisation des fichiers,
- préparation du dépôt GitHub,
- rédaction du rapport,
- préparation de la présentation.

7.2. répartition des tâches



DIFFICULTÉS RENCONTRÉES ET SOLUTIONS

Difficulté	Solution apportée
Conflits lors de la fusion du code sur Git.	Mise en place de branches par fonctionnalité et réunions de synchronisation.
Gestion des différents types d'objets dans une table unique.	Utilisation de colonnes génériques (primary_value) s'adaptant au type d'appareil.
Sécurisation des accès Admin/User.	Développement de middlewares personnalisés pour vérifier le rôle en session.

CONCLUSION ET PERSPECTIVES

À travers ce projet, nous avons pu concevoir et développer une application web complète dédiée à la gestion d'une maison intelligente. Ce travail nous a permis de mettre en pratique plusieurs notions importantes étudiées en formation, notamment le développement web, la programmation backend, la gestion de base de données, la sécurité informatique et le travail collaboratif. L'application réalisée permet la gestion des utilisateurs, la distinction entre administrateur et utilisateur standard, le contrôle de plusieurs objets connectés ainsi qu'une interface moderne et intuitive. Le système répond globalement aux besoins définis au début du projet et offre une base solide pour de futures améliorations. Au-delà de l'aspect technique, ce projet nous a également appris à travailler en équipe, à répartir les tâches, à utiliser des outils collaboratifs comme GitHub et à résoudre les problèmes rencontrés pendant le développement. Ce projet représente donc une expérience enrichissante aussi bien sur le plan académique que pratique.

Même si l'application est fonctionnelle, plusieurs évolutions peuvent être envisagées afin de la rendre encore plus performante et plus réaliste.

Améliorations techniques :

- Connexion à de véritables objets IoT (Arduino, ESP32, Raspberry Pi).
- Mise en place d'une API REST.
- Passage vers une base de données plus avancée (MySQL ou PostgreSQL).
- Hébergement en ligne de l'application.
- Sauvegarde automatique des données.

Améliorations fonctionnelles :

- Notifications en temps réel.
- Historique des actions des utilisateurs.
- Gestion de la consommation énergétique.
- Commande vocale.

- Création d'une application mobile.
- Gestion de scénarios automatiques (mode nuit, absence, réveil, etc.).

Améliorations orientées accessibilité :

- Interface adaptée aux personnes âgées.
- Assistance vocale.
- Contrôle simplifié pour les personnes à mobilité réduite.
- Téléassistance à distance pour les proches ou aidants.

Ainsi, ce projet constitue une base évolutive pouvant être enrichie pour répondre aux besoins futurs des maisons intelligentes et des utilisateurs de demain.

RÉFÉRENCES

1. **Flask Documentation** : <https://flask.palletsprojects.com/>
2. **SQLAlchemy / SQLite Guide** : <https://www.sqlite.org/docs.html>
3. **MDN Web Docs (CSS/JS)** : <https://developer.mozilla.org/>
4. **Werkzeug Security** : <https://werkzeug.palletsprojects.com/en/2.0.x/security/>
5. **supports de cours sur moodle**